

Créer des tables partitionnées et
clusterisées

Créer des tables partitionnées et clusterisées
Optimiser une requête lente
Utiliser des window functions et UDF en production
Parler concrètement de modélisation (star schema)

H1 : Partitionnement & clustering (optimisation coûts)

En tant que Data Scientist, aborder la structure des tables est un excellent réflexe. Une table bien pensée, c'est la différence entre un modèle qui s'entraîne en 2 minutes pour quelques centimes et une requête qui plante par manque de mémoire (Out of Memory) ou qui coûte des centaines d'euros à chaque itération.

Voici le guide méthodologique pour définir votre stratégie de partitionnement et de clustering sur un nouveau projet.

Étape 1 : L'audit de vos données (L'Exploratory Data Analysis - EDA)

Avant d'écrire la moindre ligne de code, vous devez analyser la "forme" de vos données lors de votre phase d'exploration. Vous cherchez deux caractéristiques clés : la cardinalité et la distribution.

1. La Cardinalité (Le nombre de valeurs uniques)

Basse cardinalité (Idéal pour le Partitionnement) : Peu de valeurs distinctes. Par exemple, des dates (365 jours par an), des mois, des pays, ou des tranches d'âges.

Haute cardinalité (Idéal pour le Clustering) : Des milliers ou millions de valeurs distinctes. Par exemple, des user_id, des codes produits, des codes postaux.

2. La Distribution (Le piège du "Data Skew")

Regardez si vos données sont équitablement réparties. Si vous partitionnez par country_code mais que 95% de vos clients sont en France, vous allez créer une partition géante (la France) et des centaines de micro-partitions vides. C'est contre-productif.

Règle d'or : Vos partitions doivent idéalement avoir une taille homogène.

Étape 2 : L'analyse de vos patterns de requêtage (Le "Query Pattern")

La structure de votre table ne dépend pas uniquement des données, elle dépend surtout de la façon dont vous allez les interroger. Regardez votre code de Feature Engineering ou vos requêtes d'extraction :

- Quels sont les filtres récurrents dans vos clauses WHERE ?
- Sur quelles colonnes faites-vous vos jointures (JOIN) ?
- Quelles sont les colonnes utilisées dans vos GROUP BY ?

Étape 3 : Choisir la bonne combinaison (Comment implémenter)

En combinant vos analyses (Données + Requêtes), vous pouvez choisir vos armes. Voici les fonctions et approches usuelles (généralement appliquées sur BigQuery ou Snowflake) :

1. Les options pour le PARTITION BY

Le partitionnement est limité (souvent un maximum de 4000 partitions par table). Vous avez trois grandes façons de le faire :

- Par Date / Temps (Le plus fréquent) :
 - Fonctions usuelles : `TIMESTAMP_TRUNC(date_column, DAY)`, `TIMESTAMP_TRUNC(date_column, MONTH)`, ou `TIMESTAMP_TRUNC(date_column, YEAR)`.
 - Quand l'utiliser : Dès que vos données ont une composante temporelle forte et que vous filtrez souvent sur les X derniers jours/mois (ex: les transactions récentes).
- Par Plages d'entiers (Integer Range) :
 - Fonctions usuelles : `RANGE_BUCKET(column, GENERATE_ARRAY(start, end, step))`.
 - Quand l'utiliser : Pour des identifiants numériques ou des étapes (comme votre colonne step dans le projet fraude).
- Par Temps d'ingestion (`_PARTITIONTIME`) :
 - La table se partitionne automatiquement selon le moment où la donnée est entrée dans le système, même s'il n'y a pas de colonne date dans vos données.

2. Les options pour le CLUSTER BY

Le clustering n'a pas de limite stricte et peut combiner jusqu'à 4 colonnes. L'ordre des colonnes compte (de la plus importante à la moins importante).

- Comment faire : On liste simplement les colonnes : `CLUSTER BY user_id, device_type`.
- Quand l'utiliser : Pour des colonnes à haute cardinalité souvent utilisées dans le `WHERE` ou le `JOIN`.

La Matrice de Décision du Data Scientist

Pour vous guider lors de votre prochain projet, voici comment trancher :

Si votre requête utilise souvent...	...Et que la colonne a une...	Alors utilisez :
Un filtre de temps (<code>date > '2026-01-01'</code>)	Cardinalité basse/moyenne	PARTITION BY (Date)
Un filtre d'identifiant (<code>user_id = 12345</code>)	Cardinalité très haute	CLUSTER BY
Une jointure fréquente (<code>A.id = B.id</code>)	Cardinalité haute	CLUSTER BY (sur les deux tables)

Un filtre sur une catégorie (type = 'TRANSFER')	Cardinalité basse, mais répartition inégale	CLUSTER BY
---	---	------------

En pratique : Le workflow sur un nouveau projet

Pour aborder cela sereinement en tant que Data Scientist :

1. Phase de dev (Petite échelle) : Travaillez sur un échantillon (LIMIT 10000). Ne vous souciez pas du partitionnement à ce stade.
2. Phase de Feature Engineering (Grande échelle) : Avant de lancer le calcul de vos features sur 5 ans d'historique, observez vos filtres. Si vous calculez des variables glissantes (ex: "nombre d'achats les 30 derniers jours"), partitionnez absolument la table finale par jour.
3. Phase de déploiement (Production) : Assurez-vous que le pipeline qui va alimenter votre modèle au quotidien (inférence) tire parti de ce partitionnement en ne lisant que la partition du jour courant.

H2 : Optimisation de requête

Pour comprendre l'impact d'une mauvaise requête sur une table partitionnée et clusterisée, rien ne vaut un cas réel.

Mettons-nous en situation : votre table raw_transactions_partitioned_clustered contient 5 ans d'historique (soit environ 500 millions de lignes, pesant 50 Go). Elle est partitionnée par jour (date_transaction) et clusterisée par type et isFraud.

Voici comment un Data Scientist junior peut accidentellement faire exploser la facture GCP, et comment le Data Engineer corrige le tir.

Cas Concret : Calculer le montant total des virements suspects survenus en 2025

❌ La version "Catastrophe" (Anti-Pattern)

Le réflexe classique est d'écrire la requête en pensant que SQL va "se débrouiller".

```
SELECT
  type,
  SUM(amount) as total_suspect_amount
FROM
  `fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`
WHERE
  EXTRACT(YEAR FROM date_transaction) = 2025 -- Piège 1
  AND LOWER(type) = 'transfer'                -- Piège 2
GROUP BY 1;
```

Pourquoi BigQuery pleure (et votre budget aussi) :

- Destruction du Partitionnement (EXTRACT) : En appliquant la fonction EXTRACT() sur la colonne date_transaction, vous forcez BigQuery à ouvrir chaque ligne de chaque partition sur les 5 ans d'historique pour calculer l'année. Le partitionnement est totalement ignoré. Données scannées : 50 Go.
- Destruction du Clustering (LOWER) : Le clustering est basé sur la valeur exacte de la chaîne ('TRANSFER'). En appliquant LOWER(type), l'index du cluster est cassé. Le moteur doit scanner tout l'intérieur de la table.

La version Optimisée (Best Practice)

Pour activer les super-pouvoirs de la table, on réécrit la clause WHERE pour qu'elle corresponde exactement à la structure physique des données.

```
SELECT
  type,
  SUM(amount) as total_suspect_amount
FROM
  `fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`
WHERE
  date_transaction BETWEEN '2025-01-01' AND '2025-12-31' -- Pruning
  actif
  AND type = 'TRANSFER' --
  Clustering actif
GROUP BY 1;
```

Pourquoi c'est ultra-efficace :

- L'Élagage (Partition Pruning) : En donnant des valeurs brutes (BETWEEN), BigQuery n'ouvre que les 365 partitions de l'année 2025. Il ignore les 4 autres années. Les données scannées tombent instantanément de 50 Go à 10 Go.
- Le Filtrage Chirurgical (Cluster Block Skipping) : À l'intérieur de ces 365 partitions, BigQuery sait que le type TRANSFER est regroupé au même endroit. Il saute tous les blocs contenant du CASH_OUT ou du PAYMENT. Les données scannées tombent de 10 Go à 2 Go.

H3 : Window functions (indispensable pour KPIs métiers)

En tant que Data Engineer, s'il y a bien un outil que vous devez fournir packagé et optimisé aux Data Scientists pour leurs KPIs ou leurs features, ce sont les Window Functions (fonctions de fenêtrage).

Leur super-pouvoir ? Elles permettent d'effectuer des calculs analytiques (moyennes glissantes, cumuls, classements) sur un groupe de lignes, sans écraser les lignes individuelles (contrairement au GROUP BY qui fusionne tout).

Sur des tables volumineuses partitionnées et clusterisées, mal maîtriser les fenêtres peut saturer la mémoire du moteur SQL (les fameuses erreurs Resources Exceeded). Voyons comment les utiliser et les optimiser à travers trois KPIs métiers indispensables.

Les 3 KPIs Métiers Clés en MLOps / Fraude

KPI 1 : La Moyenne Glissante (Feature de Détection de Fraude)

- L'objectif : Calculer le montant moyen des transactions de l'utilisateur sur ses 3 dernières transactions pour détecter un pic soudain.
- La syntaxe clé : ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

```
SELECT
    date_transaction,
    user_id,
    amount,
    AVG(amount) OVER(
        PARTITION BY user_id
        ORDER BY timestamp_transaction
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) as avg_amount_3_transactions
FROM
    `fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`
WHERE
    date_transaction = '2026-05-23'; -- Pruning actif !
```

KPI 2 : Le Cumul d'Activité (Running Total)

- **L'objectif** : Suivre l'évolution du volume financier total envoyé par un utilisateur au cours de sa journée pour vérifier s'il dépasse un plafond réglementaire (anti-blanchiment).
- **La syntaxe clé** : RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW (ou simplement omettre la clause de frame après l'ORDER BY, ce qui est le comportement par défaut).

```
SELECT
    user_id,
    timestamp_transaction,
    amount,
    SUM(amount) OVER(
        PARTITION BY user_id
        ORDER BY timestamp_transaction
    ) as cumulative_amount_today
FROM
    `fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`
WHERE
    date_transaction = '2026-05-23';
```

KPI 3 : La recherche du comportement précédent (LAG)

- **L'objectif** : Calculer le temps écoulé depuis la toute dernière transaction de cet utilisateur. Si deux transactions ont lieu à 2 secondes d'intervalle dans deux pays différents, le score de fraude explose.
- **La syntaxe clé** : LAG(column, offset)

```

SELECT
  user_id,
  timestamp_transaction,
  country,
  LAG(country, 1) OVER(
    PARTITION BY user_id
    ORDER BY timestamp_transaction
  ) as previous_country,
  TIMESTAMP_DIFF(
    timestamp_transaction,
    LAG(timestamp_transaction, 1) OVER(PARTITION BY user_id ORDER BY
timestamp_transaction),
    SECOND
  ) as seconds_since_last_tx
FROM
  `fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`
WHERE
  date_transaction = '2026-05-23';

```

KPI 4 : Requête de détection de Smurfing

Les fraudeurs connaissent les plafonds de détection des banques (ex: "alerte automatique au-dessus de 10 000 €"). Pour contourner cela, ils font du "schtroumpfage" (smurfing) : couper une grosse somme en plusieurs petites sommes envoyées très rapidement.

Pour détecter cela, il faut coupler votre steps_since_last_tx avec une autre Window Function qui calcule la somme cumulée ou le nombre d'occurrences.

```

WITH velocity_analysis AS (
  SELECT
    nameOrig,
    step,
    type,
    amount,
    -- Votre indicateur corrigé
    IFNULL(step - LAG(step, 1) OVER(PARTITION BY nameOrig ORDER BY
step), -1) as steps_since_last_tx,
    -- KPI d'appui : Combien de transactions d'affilée dans la même
heure ?
    COUNT(*) OVER(PARTITION BY nameOrig, step) as nb_tx_same_hour,
    -- KPI d'appui : Somme cumulée sur les 3 dernières transactions
    SUM(amount) OVER(PARTITION BY nameOrig ORDER BY step ROWS BETWEEN 2
PRECEDING AND CURRENT ROW) as rolling_sum_3_tx
  FROM
    `fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`

```

```

WHERE
    step BETWEEN 100 AND 200
)
SELECT *
FROM velocity_analysis
WHERE steps_since_last_tx = 0 AND nb_tx_same_hour >= 3;

```

Le Guide d'Optimisation du Data Engineer

Le fenêtrage est extrêmement gourmand en ressources car le moteur SQL doit trier les données en mémoire pour appliquer le PARTITION BY de la fenêtre. Voici comment coupler l'efficacité du partitionnement de table avec les Window Functions :

1. Le faux ami : Aligner le PARTITION BY de la table et de la fenêtre

- **Le piège** : On pourrait penser que faire un PARTITION BY date_transaction dans la fenêtre est une bonne idée. C'est souvent inutile si vous avez déjà filtré votre table dans le WHERE.
- **La règle** : Utilisez le WHERE pour restreindre physiquement les données lues par BigQuery, puis utilisez le PARTITION BY de la fenêtre uniquement sur votre entité métier (ex: user_id, account_id).

2. Le Clustering à la rescousse

Si vous savez que vos requêtes de KPIs métiers vont faire un PARTITION BY user_id dans leur fenêtre de calcul, ajoutez user_id dans vos colonnes de CLUSTER BY lors de la création de la table.

- Pourquoi ? BigQuery va stocker les lignes du même utilisateur physiquement proches les unes des autres. Lorsque la Window Function va s'exécuter, le moteur n'aura presque aucun effort de tri à faire en mémoire. La requête s'exécutera à la vitesse de l'éclair.

3. Attention au "Data Skew" en mémoire

Si un utilisateur (par exemple, un robot ou un marchand mondial) possède 10 millions de transactions à lui seul, le PARTITION BY user_id de la fenêtre va envoyer ces 10 millions de lignes sur un seul nœud de calcul en mémoire, provoquant un crash de la requête.

- La parade : Si vous suspectez la présence de "gros comptes", couplez votre partition de fenêtre avec une dimension temporelle plus fine, comme l'heure ou la demi-journée (PARTITION BY user_id, TIMESTAMP_TRUNC(timestamp_transaction, HOUR)).

En résumé : L'anatomie d'une Window Function parfaite

FONCTION() OVER (PARTITION BY [Clustered Column] ORDER BY [Time Column] ROWS/RANGE [Frame])

▲	▲	▲	▲
Qu'est-ce qu'on calcule ?	Sur qui on regroupe ?	Dans quel ordre ?	Sur quelle largeur de vue ?

H4 : UDF (User Defined Functions) et scripts

Pour clore notre stack MLOps de pointe, il nous reste une arme secrète pour automatiser et nettoyer nos détections d'anomalies : les UDF (User-Defined Functions) et les Scripts (Procédures stockées).

En tant que Data Engineer / Data Scientist, écrire des requêtes de 200 lignes pleines de calculs répétitifs (comme vos corrections de NULL ou vos calculs de scores de fraude) est une mauvaise pratique. Les UDF permettent de packager votre logique métier, tandis que les scripts permettent d'enchaîner les opérations directement dans BigQuery.

Voyons comment les implémenter sur notre projet de fraude.

1. Les UDF (User-Defined Functions) : Packager la logique métier

Une UDF est une fonction personnalisée que vous créez en SQL ou en JavaScript, et que vous pouvez réutiliser directement dans vos requêtes.

Cas concret : Créer un "Score d'agressivité temporelle"

Plutôt que d'écrire un énorme bloc CASE WHEN dans chaque requête de feature engineering, on va créer une UDF SQL réutilisable par toute l'équipe.

```
-- 1. Création de la fonction (à ne lancer qu'une seule fois)
CREATE OR REPLACE FUNCTION `fraud_detection.GET_TEMPORAL_SCORE` (
  steps_since_last_tx INT64,
  tx_type STRING,
  amount FLOAT64
) AS (
  CASE
    -- Cas 1 : Vitesse critique sur virement/retrait (Risque Maximal)
    WHEN steps_since_last_tx = 0 AND tx_type IN ('TRANSFER', 'CASH_OUT')
  THEN 0.95
    -- Cas 2 : Fractionnement suspect (Risque Élevé)
    WHEN steps_since_last_tx = 0 AND amount > 5000 THEN 0.80
    -- Cas 3 : Comportement rapide sur petit paiement (Risque Faible)
    WHEN steps_since_last_tx = 0 THEN 0.30
    -- Cas 4 : Première transaction ou délai normal
    WHEN steps_since_last_tx = -1 THEN 0.10
    ELSE 0.0
  END
);
```

Comment l'utiliser dans votre requête optimisée ?

Une fois créée, votre UDF s'appelle aussi simplement qu'une fonction native (SUM ou AVG). Votre code devient d'une clarté absolue :

```
SELECT
  nameOrig,
  step,
  type,
```

```

amount,
-- 2. Appel de la fonction avec nos variables
`fraud_detection.GET_TEMPORAL_SCORE` (
    IFNULL(step - LAG(step, 1) OVER(PARTITION BY nameOrig ORDER BY
step), -1),
    type,
    amount
) as fraud_behavior_score
FROM
`fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`
WHERE
    step BETWEEN 100 AND 200;

```

2. Les Scripts et Procédures Stockées : L'industrialisation

Un Script (souvent encapsulé dans une PROCEDURE) permet d'exécuter plusieurs instructions SQL à la suite (déclarer des variables, créer des tables temporaires, faire des boucles).

C'est l'outil parfait pour votre pipeline quotidien : au lieu qu'Airflow envoie 4 requêtes différentes, il appelle une seule procédure sur BigQuery, ce qui limite les allers-retours réseau et centralise la logique.

Cas concret : Le script quotidien de Feature Engineering

Voici le script exact que vous pouvez programmer pour s'exécuter chaque nuit afin d'alimenter votre modèle ML avec les données du dernier "bucket" de steps :

```

CREATE OR REPLACE PROCEDURE
`fraud_detection.UPDATE_DAILY_FEATURES` (target_step INT64)
BEGIN
    -- Déclaration de variables locales pour le script
    DECLARE min_bucket INT64;
    DECLARE max_bucket INT64;

    -- On définit notre fenêtre de calcul autour du step cible (ex:
fenêtrage de sécurité)
    SET min_bucket = target_step - 10;
    SET max_bucket = target_step;

    -- Étape 1 : Création d'une table temporaire de features avec nos UDFs
    CREATE OR REPLACE TEMP TABLE tmp_features AS
    SELECT
        nameOrig,
        step,
        type,
        amount,

```

```

    `fraud_detection.GET_TEMPORAL_SCORE` (
        IFNULL(step - LAG(step, 1) OVER(PARTITION BY nameOrig ORDER BY
step), -1),
        type,
        amount
    ) as calculated_score
FROM

`fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`
WHERE
    step BETWEEN min_bucket AND max_bucket; -- Pruning actif dynamique !

-- Étape 2 : Insertion des résultats filtrés dans la table finale pour
le modèle ML
INSERT INTO `fintec-496811.fraud_detection.model_features_output`
SELECT * FROM tmp_features WHERE step = target_step;

END;

```

Pour exécuter ce pipeline complet, votre orchestrateur (Airflow ou Dataform) n'a plus qu'à lancer cette ligne unique :

```
CALL `fraud_detection.UPDATE_DAILY_FEATURES` (150);
```

Ce qu'il faut retenir pour votre profil de Data Engineer

- **UDF en SQL vs JavaScript** : Privilégiez toujours le SQL pour vos UDF. Les UDF JavaScript sont puissantes (elles permettent d'embarquer des mini-librairies), mais elles s'exécutent en dehors du moteur natif de BigQuery, ce qui ralentit fortement les requêtes sur des millions de lignes.
- **Maintenance** : Centraliser vos règles métiers (définition d'une fraude, d'un client actif, d'un compte dormant) dans des UDF partagées évite que le Data Scientist A et le Data Scientist B utilisent deux définitions différentes dans leurs coins.
- **Couplage avec l'architecture** : En combinant l'élagage des partitions (WHERE step BETWEEN...) à l'intérieur de vos procédures stockées, vous créez des micro-moteurs de calcul asynchrones, isolés et ultra-économiques.

H5 : Modélisation star schema

Pourquoi faire un Star Schema ici ? Parce que votre table d'origine raw_transactions est "dénormalisée" (plate et géante). Elle contient à la fois des données de flux (montants) et des données d'entités (noms des clients, comptes). En production, cela crée de la redondance et alourdit les calculs.

Voici le plan de notre mini-projet : isoler le Fait (la transaction) des Dimensions (les acteurs), tout en conservant notre optimisation par partitionnement et clustering.

1. L'Architecture du Star Schema

L'objectif est de diviser notre base en deux types de tables :

- La Table de Faits (fact_transactions) : Elle contient les événements (les métriques quantifiables, les montants) et les clés étrangères. Elle est immense et grandit chaque jour.
- Les Tables de Dimensions (dim_users, dim_steps) : Elles contiennent le contexte (les attributs, les profils). Elles sont plus petites.

2. Étape 1 : Création de la Dimension Utilisateurs (dim_users)

Cette table regroupe l'historique et le profil des comptes. Comme un utilisateur peut être à la fois émetteur (nameOrig) et destinataire (nameDest), on centralise tout.

```
CREATE OR REPLACE TABLE `fraud_detection.dim_users`  
CLUSTER BY user_id AS  
SELECT  
  -- Génération d'un identifiant unique propre pour le modèle  
  ROW_NUMBER() OVER() as user_id,  
  user_name,  
  -- Métadonnées utiles pour le ML (première et dernière fois vus)  
  MIN(min_step) as first_active_step,  
  MAX(max_step) as last_active_step  
FROM (  
  SELECT nameOrig as user_name, MIN(step) as min_step, MAX(step) as  
max_step FROM `fraud_detection.raw_transactions` GROUP BY 1  
  UNION DISTINCT  
  SELECT nameDest as user_name, MIN(step) as min_step, MAX(step) as  
max_step FROM `fraud_detection.raw_transactions` GROUP BY 1  
)  
GROUP BY user_name;
```

3. Étape 2 : Création de la Table de Faits (fact_transactions)

C'est ici que l'on applique notre stratégie de performance. Cette table va lier les transactions aux clés de notre dimension dim_users via des jointures, tout en restant partitionnée par step et clusterisée par type.

```
CREATE OR REPLACE TABLE `fraud_detection.fact_transactions`  
PARTITION BY  
  RANGE_BUCKET(step, GENERATE_ARRAY(0, 1000, 100))  
CLUSTER BY  
  type, isFraud AS  
SELECT  
  t.step,  
  t.type,  
  t.amount,  
  -- Remplacement des chaînes de caractères lourdes par nos clés  
numériques légères  
  orig.user_id as orig_user_id,
```

```

    dest.user_id as dest_user_id,
    t.oldbalanceOrig,
    t.newbalanceOrig,
    t.oldbalanceDest,
    t.newbalanceDest,
    t.isFraud,
    t.isFlaggedFraud
FROM
    `fraud_detection.raw_transactions` t
JOIN
    `fraud_detection.dim_users` orig ON t.nameOrig = orig.user_name
JOIN
    `fraud_detection.dim_users` dest ON t.nameDest = dest.user_name;

```

4. Étape 3 : Requêtage et Calcul de KPIs sur le Star Schema

Maintenant que notre modèle en étoile est prêt, voyons comment un Data Scientist extrait ses features de vélocité de manière ultra-rapide. On réutilise notre Window Function, mais cette fois sur la table de faits optimisée.

```

WITH optimized_features AS (
    SELECT
        step,
        orig_user_id,
        type,
        amount,
        -- Fenêtrage ultra-rapide sur l'ID numérique au lieu de la chaîne de
        -- caractères
        IFNULL(step - LAG(step, 1) OVER(PARTITION BY orig_user_id ORDER BY
        step), -1) as steps_since_last_tx
    FROM
        `fraud_detection.fact_transactions`
    WHERE
        step BETWEEN 100 AND 200 -- L'élégage physique de la table de faits
        est actif
)
SELECT
    f.step,
    u.user_name, -- On récupère le nom uniquement à la fin pour
    l'affichage
    f.type,
    f.amount,
    f.steps_since_last_tx,
    -- Appel de notre UDF de score d'agressivité
    `fraud_detection.GET_TEMPORAL_SCORE`(f.steps_since_last_tx, f.type,
    f.amount) as fraud_score
FROM

```

```
optimized_features f
JOIN
  `fraud_detection.dim_users` u ON f.orig_user_id = u.user_id
WHERE
  f.steps_since_last_tx = 0; -- Ciblage des anomalies métiers
```

Le Bilan d'Architecture pour l'équipe

En passant d'une table plate à ce Star Schema optimisé, vous avez débloqué trois gains majeurs :

- Volume de stockage réduit : Remplacer les colonnes de texte nameOrig et nameDest (ex: "C123456789") par des entiers INT64 (user_id) dans la table de faits réduit le poids de la table de près de 30%.
- Vitesse de tri démultipliée : Le moteur BigQuery trie et partitionne les entiers numériques infiniment plus vite que les chaînes de texte. Votre Window Function PARTITION BY orig_user_id s'exécute en un clin d'œil.
- Évolutivité (Scalability) : Si demain le Data Scientist veut ajouter des features sur l'utilisateur (ex: l'âge du compte, le pays d'origine), il lui suffit de modifier la table dim_users. La table de faits, elle, ne bouge pas.

Dataflow & Cloud Composer (Airflow sur GCP)

Dataflow & Cloud Composer (Airflow sur GCP)

- Comprendre ce qu'est Apache Beam / Dataflow (batch & streaming)
- Écrire un pipeline simple (logique) même si tu n'exécutes pas en production
- Créer un DAG Airflow fonctionnel sur Cloud Composer
- Orchestrer une tâche BigQuery + un job Dataflow

H1 : Comprendre Dataflow & Apache Beam (sans coder tout de suite)

Jusqu'à présent, nous avons fait subir à BigQuery des calculs analytiques lourds (nos fameuses Window Functions et requêtes d'anomalies). C'est l'approche ELT (Extract, Load, Transform) : on charge tout en vrac dans l'entrepôt de données, puis on trie.

Mais que se passe-t-il si vos transactions arrivent en continu, seconde après seconde, et que vous devez détecter la fraude avant même qu'elle ne soit écrite dans BigQuery ? C'est là qu'entrent en scène Apache Beam et Cloud Dataflow, pour basculer dans le monde de l'ETL pipeline (Extract, Transform, Load) et du temps réel.

1. Qu'est-ce qu'Apache Beam ? (Le Cerveau)

Apache Beam est un modèle de programmation open-source (un SDK disponible en Python, Java et Go). C'est le code que vous écrivez.

Son nom est un mot-valise très révélateur : Batch + Stream = Beam.

- **Le concept unifié** : Avant Apache Beam, si vous vouliez faire du traitement par lot (Batch - ex: analyser les logs de la semaine dernière), vous deviez écrire du code MapReduce ou Spark. Si vous vouliez faire du temps réel (Streaming - ex: analyser les transactions à la volée), vous deviez réécrire un code totalement différent avec Storm ou Flink.
- **La promesse de Beam** : Vous écrivez votre logique algorithmique une seule fois. Que la donnée d'entrée soit un fichier fixe de 10 Go (Batch) ou un flux ininterrompu de messages Pub/Sub (Streaming), le code reste exactement le même.

2. Qu'est-ce que Cloud Dataflow ? (Les Muscles)

Si Apache Beam est le code (la recette), Cloud Dataflow est l'infrastructure managée de GCP qui va exécuter ce code (la cuisine).

Dataflow est un moteur d'exécution totalement serverless et hautement parallélisé.

- **L'Auto-scaling horizontal** : Vous lancez votre pipeline de détection de fraude. À 3h du matin, il y a 10 transactions par seconde : Dataflow fait tourner un seul petit serveur (Worker). À 14h, lors du pic d'achats, le flux passe à 50 000 transactions par seconde : Dataflow démarre automatiquement 50 serveurs en parallèle, découpe le flux de données, répartit la charge, puis éteint les serveurs quand le pic est passé.
- **L'optimisation dynamique (Dynamic Work Rebalancing)** : Si un serveur ralentit (l'effet de bord du Data Skew dont on parlait pour les fenêtres SQL), Dataflow détecte le goulot d'étranglement, retire une partie du travail à ce serveur et la redistribue aux autres serveurs disponibles. Vous ne gérez aucune infrastructure.

3. Le Lexique Indispensable d'Apache Beam

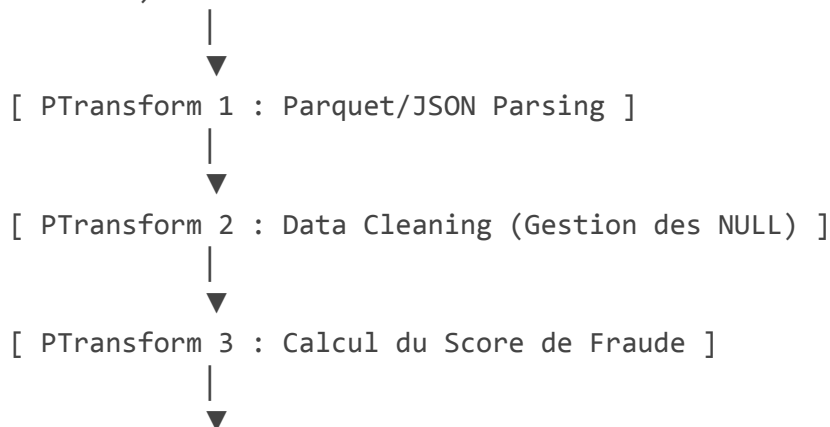
Pour concevoir un pipeline mentalement, vous devez maîtriser les 4 piliers du modèle Beam :

- Le Pipeline : C'est le conteneur global, l'ensemble de votre voyage de données, de la source à la destination.
- La PCollection (Parallel Collection) : C'est la donnée qui circule. Contrairement à une liste Python classique en mémoire, une PCollection est une collection de données potentiellement infinie, immuable, et surtout distribuée sur plusieurs machines.
- Le PTransform (Parallel Transform) : C'est l'opération de calcul appliquée à une PCollection. Un PTransform prend une donnée en entrée, applique une logique (ex: filtrer, nettoyer, calculer un score), et recrée une nouvelle PCollection modifiée en sortie.
- Le Runner : C'est le moteur qui traduit votre code pour l'infrastructure cible. En phase de test sur votre PC, vous utiliserez le DirectRunner (votre processeur local). En production sur GCP, vous utiliserez le DataflowRunner.

4. La Vision Mentale du Pipeline de Fraude

Visualisons comment se structure logiquement un pipeline Beam pour notre projet FinTech :

[Source : Pub/Sub ou Cloud Storage] <-- (Entrée : PCollection de lignes brutes)



[Sink (Destination) : BigQuery Table] <-- (Sortie : Écrit dans la bonne partition)

H2 : Premier pipeline Beam (sur notebook, sans GCP)

Pour construire notre premier pipeline Apache Beam sans toucher à l'infrastructure cloud, nous allons utiliser la syntaxe Python de Beam en mode local. Dans un notebook Google Colab, cela se fait très simplement en utilisant le DirectRunner. Ce moteur d'exécution va simuler un cluster de calcul directement en utilisant les cœurs de processeur alloués à votre notebook.

Voici la structure exacte et le code logique à copier-coller dans votre Colab pour nettoyer nos données de transactions et calculer notre premier indicateur.

1. Préparation de l'environnement (Cellule 1)

Avant d'écrire le pipeline, il faut installer le SDK Apache Beam dans l'environnement Colab.

```
# Installe le package Apache Beam
!pip install apache-beam[gcp] --quiet
```

2. Le Code du Pipeline Logique (Cellule 2)

La syntaxe d'Apache Beam utilise l'opérateur pipe (|) de manière très spécifique. Au lieu d'écrire des fonctions imbriquées, le | sert à passer une PCollection à un PTransform. C'est l'équivalent visuel d'une flèche de flux : Donnée | "Nom de l'étape" >> Transformation.

Voici le script complet à exécuter :

```
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions
import json

# --- 1. DÉFINITION DES COMPOSANTS LOGIQUES (DoFn) ---

class CleanAndParseTransaction(beam.DoFn):
    """
    Cette classe hérite de DoFn (Dofunction). C'est le cœur de notre
    PTransform.
    Elle prend une ligne de texte (JSON ou CSV), la nettoie et gère les
    anomalies techniques (NULL).
    """
    def process(self, element):
        try:
            # En production, la donnée arrive souvent sous forme de
            chaîne JSON
            data = json.loads(element)

            # Correction des anomalies techniques / Valeurs par défaut
            step = int(data.get('step', -1))
            tx_type = str(data.get('type', 'UNKNOWN')).upper()
            amount = float(data.get('amount', 0.0))
            nameOrig = str(data.get('nameOrig', 'MISSING'))

            # Filtrage logique : On ne garde que les données valides
            if step != -1 and nameOrig != 'MISSING':
                # On renvoie un dictionnaire propre (le rendement sous
                forme de générateur)
                yield {
                    'step': step,
                    'type': tx_type,
                    'amount': amount,
                    'nameOrig': nameOrig
                }
        except:
            pass
```

```

        except Exception as e:
            # En production, on enverrait ces lignes corrompues dans une
            Dead-Letter Queue (DLQ)
            pass

```

```

class FilterSuspiciousAmount(beam.DoFn):
    """Filtrage métier : On isole les transactions suspectes à fort
    montant"""
    def process(self, element):
        if element['amount'] > 100000.0: # Seuil arbitraire de
suspicion
            yield element

```

--- 2. EXÉCUTION DU PIPELINE LOCAL ---

```

# Configuration des options pour un run local
options = PipelineOptions()

```

```

# Simulation de données d'entrée brutes (comme si elles venaient d'un
fichier)

```

```

raw_mock_data = [
    '{"step": 120, "type": "transfer", "amount": 540000.0, "nameOrig":
"C1001"}',
    '{"step": 120, "type": "payment", "amount": 15.40, "nameOrig":
"C1002"}',
    '{"step": 121, "type": "cash_out", "amount": 850000.0, "nameOrig":
"C1003"}',
    '{"step": -1, "type": "transfer", "amount": 100.0, "nameOrig":
"MISSING"}' # Devrait être rejeté
]

```

```

# Le bloc 'with' initialise et ferme proprement le pipeline à la fin du
traitement

```

```

with beam.Pipeline(runner='DirectRunner', options=options) as pipeline:

```

```

    # Étape A : Création de la PCollection initiale

```

```

    input_data = (
        pipeline
        | "Créer les données" >> beam.Create(raw_mock_data)
    )

```

```

    # Étape B : Nettoyage et parsing

```

```

    cleaned_data = (
        input_data
        | "Nettoyer & Parser" >> beam.ParDo(CleanAndParseTransaction())
    )

```

```

)

# Étape C : Filtrage métier
suspicious_data = (
    cleaned_data
    | "Isoler Gros Montants" >> beam.ParDo(FilterSuspiciousAmount())
)

# Étape D : Restitution (Affichage dans la console Colab)
# En production, beam.Map(print) serait remplacé par une écriture
dans BigQuery
suspicious_data | "Afficher les alertes" >> beam.Map(print)

```

3. Analyse de la syntaxe : Comprendre le code

- `beam.Create()` : C'est notre source d'entrée (Source). Elle transforme une simple liste Python en une PCollection Beam distribuable.
- `beam.ParDo()` : Signifie Parallel Do. C'est le moteur de transformation de Beam. Il prend une classe DoFn et l'applique à chaque élément de la PCollection de manière indépendante et parallèle.
- `yield` vs `return` : Dans une classe DoFn, on utilise toujours `yield`. Cela permet à une seule ligne d'entrée de générer 0 élément (si elle est filtrée), 1 élément (si elle est transformée), ou plusieurs éléments (si on duplique la donnée), sans charger toute la mémoire d'un coup.

4. Ce qu'il se passe à l'exécution dans Colab

Lorsque vous lancez cette cellule, le `DirectRunner` va compiler votre code, construire le graphe de dépendances (DAG) et exécuter les étapes en séquence.

Dans la console, vous ne verrez s'afficher que les lignes qui ont survécu au nettoyage ET au filtre de montant :

```

{'step': 120, 'type': 'TRANSFER', 'amount': 540000.0, 'nameOrig': 'C1001'}
{'step': 121, 'type': 'CASH_OUT', 'amount': 850000.0, 'nameOrig': 'C1003'}

```

La ligne avec le `step -1` a été nettoyée, et la ligne à 15.40€ a été filtrée.

Vous venez de concevoir votre premier pipeline ETL. La beauté de la chose ? Pour exécuter ce même code sur des pétaoctets de données réelles sur GCP, il suffira de changer deux lignes de configuration pour cibler le *DataflowRunner* et une table BigQuery.

H3 : Pipeline Beam avec BigQuery (simulé ou réel)

Pour faire passer notre pipeline de la simulation locale vers la production, nous devons maintenant le connecter à BigQuery, notre destination finale (Sink).

Dans le modèle Apache Beam, la connexion à BigQuery est native grâce au module `beam.io.gcp.bigquery`. Ce connecteur s'occupe de tout : il convertit vos dictionnaires Python au format attendu par BigQuery et gère l'insertion par lots (Batch) ou en continu (Streaming).

Voici comment structurer le code pour lire depuis une source et écrire directement dans notre table partitionnée et clusterisée.

1. Le Code du Pipeline Connecté à BigQuery

Ce script est conçu pour s'exécuter sur GCP (ou localement si vous avez configuré vos identifiants Google Cloud). Il utilise le connecteur `WriteToBigQuery`.

```
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions,
GoogleCloudOptions
import json

# --- 1. CONFIGURATION DES OPTIONS DE PRODUCTION ---

options = PipelineOptions()

# Configuration spécifique à Google Cloud
gcp_options = options.view_as(GoogleCloudOptions)
gcp_options.project = 'fintec-496811'
gcp_options.region = 'europe-west1' # Toujours spécifier la région de
calcul
gcp_options.job_name = 'template-fraud-detection-pipeline'
gcp_options.staging_location = 'gs://fintec-496811-bucket/staging'
gcp_options.temp_location = 'gs://fintec-496811-bucket/temp'

# Définition de la table cible
target_table =
'fintec-496811:fraud_detection.raw_transactions_partitioned_clustered'

# --- 2. LOGIQUE DE TRANSFORMATION (DoFn) ---

class ProcessAndFormatForBQ(beam.DoFn):
    """Nettoie la donnée et renvoie un dictionnaire STRICTEMENT aligné
    avec le schéma BQ"""
    def process(self, element):
        try:
            data = json.loads(element)
```

```

# On génère des dictionnaires dont les clés correspondent
EXACTEMENT
# aux noms des colonnes de notre table BigQuery
yield {
    'step': int(data.get('step')),
    'type': str(data.get('type')).upper(),
    'amount': float(data.get('amount')),
    'nameOrig': str(data.get('nameOrig')),
    'oldbalanceOrig': float(data.get('oldbalanceOrig', 0.0)),
    'newbalanceOrig': float(data.get('newbalanceOrig',
0.0)),

    'nameDest': str(data.get('nameDest')),
    'oldbalanceDest': float(data.get('oldbalanceDest',
0.0)),

    'newbalanceDest': float(data.get('newbalanceDest',
0.0)),

    'isFraud': int(data.get('isFraud', 0)),
    'isFlaggedFraud': int(data.get('isFlaggedFraud', 0))
}
except Exception as e:
    # En production : envoi de la ligne en erreur vers un bucket
d'anomalies (DLQ)
    pass

```

```

# --- 3. EXÉCUTION DU PIPELINE ---

```

```

# Simulation de l'arrivée d'un fichier dans Cloud Storage (Batch)
input_gcs_path =
'gs://fintec-496811-bucket/incoming_raw_transactions/*.json'

with beam.Pipeline(runner='DataflowRunner', options=options) as
pipeline:

    (
        pipeline
        # Étape 1 : Lecture des fichiers textes bruts depuis GCS
        | "Lire depuis GCS" >> beam.io.ReadFromText(input_gcs_path)

        # Étape 2 : Parsing et structuration des données
        | "Formater pour BigQuery" >>
beam.ParDo(ProcessAndFormatForBQ())

        # Étape 3 : Écriture dans la table optimisée
        | "Écrire dans BigQuery" >> beam.io.WriteToBigQuery(

```

```

        table=target_table,
        # Le mode CREATE_IF_NEEDED évite de faire planter le job si
la table n'existe pas

create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED,
        # WRITE_APPEND signifie qu'on ajoute les lignes à la suite
(indispensable pour l'incrémental)
        write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND
    )
)

```

2. Décryptage Mécanique : Comment Beam respecte le Partitionnement ?

C'est ici que la magie du couplage Dataflow avec BigQuery opère.

Votre table cible est configurée avec PARTITION BY RANGE_BUCKET(step, ...). Lors de l'exécution de l'étape WriteToBigQuery, Apache Beam ne va pas insérer les données en vrac au hasard.

- Le découpage intelligent : Dataflow va analyser les dictionnaires qu'il a en mémoire. S'il traite un lot de transactions avec des step variant de 50 à 250, il va regrouper les lignes par "destination physique".
- L'insertion ciblée : Le connecteur BigQuery de Beam va envoyer les données en ouvrant spécifiquement les buckets de partitions concernés (0-100, 100-200, 200-300).
- Le respect du Clustering : Une fois les lignes injectées dans leurs partitions respectives, BigQuery prend le relais en arrière-plan pour réorganiser les blocs selon vos critères (CLUSTER BY type, isFraud).

3. Le point d'attention : Batch vs Streaming

Le code ci-dessus utilise beam.io.ReadFromText, c'est un mode Batch. Dataflow va s'allumer, traiter le fichier, l'écrire dans BigQuery, puis s'éteindre.

Si vous décidez de passer en Streaming (temps réel) en remplaçant la source par un flux de messagerie direct (beam.io.ReadFromPubSub) :

- Le code reste globalement identique.
- Dataflow restera allumé en continu (24h/24).
- WriteToBigQuery basculera automatiquement en mode Streaming Inserts. La donnée sera disponible dans BigQuery dans la seconde qui suit sa création, prête à être analysée par vos requêtes de fenêtrage.

Maintenant que notre pipeline de traitement sait lire et écrire au bon endroit, il faut automatiser son déclenchement au sein d'un workflow global.

H4 : Comprendre Cloud Composer (Airflow)

Maintenant que nous avons notre code SQL optimisé dans BigQuery et notre pipeline ETL prêt à tourner sur Dataflow, une question cruciale se pose : qui s'occupe de lancer tout ça au bon moment, dans le bon ordre, et de nous alerter si un calcul échoue ?

C'est exactement le rôle de Cloud Composer, la brique d'orchestration de Google Cloud.

1. Qu'est-ce que Cloud Composer ?

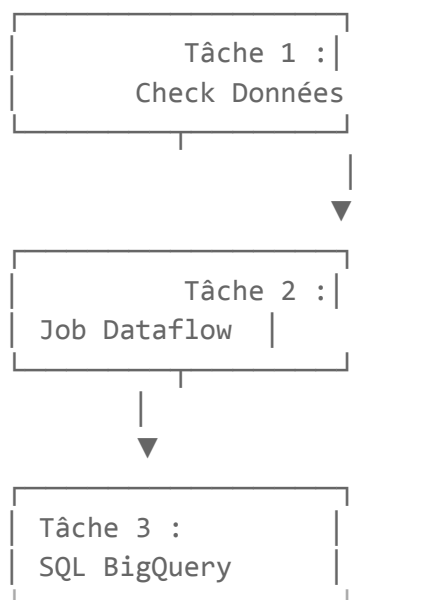
Cloud Composer est la version entièrement managée d'Apache Airflow par Google.

- **Le concept d'Airflow** : C'est un outil open-source où vous définissez des flux de travail (workflows) sous forme de code Python. On appelle ces flux des DAG (Directed Acyclic Graphs, ou Graphes Orientés Acycliques).
- **Le rôle de Google (Managed)** : Faire tourner Airflow à la main demande de gérer des serveurs, des bases de données pour l'historique, et des gestionnaires de files d'attente (Celery/Kubernetes). Cloud Composer vous affranchit de tout cela : en quelques clics, Google déploie un cluster sécurisé et prêt à l'emploi.

2. Qu'est-ce qu'un DAG ? (Le squelette du workflow)

Un DAG est un ensemble de tâches que l'on veut exécuter, structuré selon des dépendances strictes.

- Orienté : Le flux a un sens précis (de la tâche A vers la tâche B).
- Acyclique : Il est strictement interdit d'avoir des boucles infinies (la tâche B ne peut pas réactiver la tâche A). Le flux doit avoir un début et une fin.



3. Les 3 Piliers du code Airflow

Pour écrire un DAG, un Data Engineer manipule trois concepts fondamentaux en Python :

A. Le DAG (Le conteneur)

C'est le bloc principal qui définit les règles du jeu : quand le flux doit-il démarrer (`start_date`), à quelle fréquence s'exécuter (`schedule_interval`), et que faire en cas d'échec (ex: envoyer un e-mail).

B. Les Operators (Les actions)

Un opérateur est une classe Python pré-intégrée qui sait exécuter une action spécifique. C'est la nature de votre tâche. Sur GCP, on utilise les opérateurs officiels de Google :

- *BigQueryExecuteQueryOperator* : Pour lancer une requête SQL, appeler une procédure ou mettre à jour une partition.
- *DataflowTemplatedJobStartOperator* : Pour déclencher un pipeline de calcul Dataflow.
- *CloudStorageToBigQueryOperator* : Pour charger un fichier CSV brut directement dans une table de staging.

C. Les Task Dependencies (Le chaînage)

Une fois les opérateurs définis, on utilise l'opérateur de décalage binaire (`>>`) pour lier les tâches entre elles et dessiner le graphe.

- Exemple : `check_files >> run_dataflow >> update_bigquery`

4. Pourquoi utiliser Cloud Composer plutôt que des scripts planifiés (Cron) ?

Le vieux réflexe système consiste à mettre un script "Cron" sur un serveur pour lancer les requêtes toutes les nuits à 2h du matin. En production MLOps, c'est extrêmement dangereux. Cloud Composer résout tous les problèmes du Cron :

1. La gestion des dépendances : Si votre fichier de transactions arrive avec 2 heures de retard, le Cron va quand même lancer le modèle ML sur une table vide. Airflow, lui, sait attendre que l'étape précédente soit validée.

2. La reprise sur panne (Idempotence & Rerun) : Si la tâche 3 échoue à cause d'un problème réseau, Airflow vous permet de cliquer sur un bouton pour relancer uniquement la tâche 3, sans avoir à réexécuter tout le pipeline depuis le début.

3. La visibilité (UI) : Vous disposez d'une interface graphique complète pour voir quelles tâches ont réussi, lesquelles ont échoué, et consulter les logs d'exécution en un clic.

H5 : Écrire ton premier DAG Airflow (local ou simulé)

Pour comprendre la mécanique d'Airflow sans avoir à déployer un cluster Cloud Composer complet (qui prend du temps et génère des coûts), nous allons écrire un DAG en utilisant la syntaxe Python standard d'Airflow.

Ce code est parfaitement valide : vous pouvez le tester localement sur votre machine (via un simple pip install apache-airflow) ou le déposer directement dans le dossier /dags de Cloud Composer en production.

1. Le Code Complet du DAG Logique

Voici notre premier workflow. Il va simuler notre pipeline MLOps de détection de fraude en enchaînant trois étapes clés : vérifier la présence des données brutes, lancer notre

traitement d'ingestion/nettoyage, puis exécuter notre requête SQL optimisée de calcul de KPIs.

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.empty import EmptyOperator
from airflow.operators.python import PythonOperator

# --- 1. FONCTIONS PYTHON (Simulations de nos briques techniques) ---

def check_incoming_transactions():
    """Simule la vérification de l'arrivée de fichiers sur Cloud
    Storage"""
    print("Vérification du bucket
    gs://fintec-496811-bucket/incoming_raw_transactions/...")
    print("Statut : Fichiers JSON détectés. Le pipeline peut
    continuer.")

def run_fraud_dataflow_pipeline():
    """Simule le déclenchement de notre pipeline Apache Beam sur
    Dataflow"""
    print("Démarrage du job Dataflow :
    template-fraud-detection-pipeline...")
    print("Traitement en cours... Nettoyage des NULL accompli.")
    print("Statut : Données injectées avec succès dans la table
    partitionnée.")

# --- 2. CONFIGURATION DU WORKFLOW (Le DAG) ---

default_args = {
    'owner': 'data_team',
    'depends_on_past': False,
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 2,                                # En cas d'échec, Airflow
    réessaiera 2 fois
    'retry_delay': timedelta(minutes=5),         # Attendre 5 minutes entre
    chaque tentative
}

# Initialisation du conteneur global
with DAG(
    dag_id='mlops_fraud_pipeline_local',      # L'identifiant unique du DAG
    dans l'UI Airflow
    default_args=default_args,
```

```

description='Pipeline quotidien d'extraction des features de
fraude',
schedule_interval='@daily',          # S'exécute automatiquement
toutes les nuits
start_date=datetime(2026, 5, 1),      # Date de début historique du
workflow
catchup=False,                        # Désactive le backfill
automatique du passé au déploiement
tags=['fraud', 'mlops'],
) as dag:

```

```

# --- 3. DÉFINITION DES TÂCHES (Les Operators) ---

```

```

# Tâche d'entrée de jeu (Pratique pour marquer le début visuel du
graphe)

```

```

start_pipeline = EmptyOperator(
    task_id='start_pipeline'
)

```

```

# Étape 1 : Vérification

```

```

task_check_data = PythonOperator(
    task_id='check_gcs_data',
    python_callable=check_incoming_transactions
)

```

```

# Étape 2 : Lancement ETL

```

```

task_run_dataflow = PythonOperator(
    task_id='trigger_dataflow_etl',
    python_callable=run_fraud_dataflow_pipeline
)

```

```

# Étape 3 : Calcul des features (Simulation SQL)

```

```

task_run_bigquery_kpis = PythonOperator(
    task_id='execute_bigquery_window_functions',
    python_callable=lambda: print("Appel de la procédure : CALL
fraud_detection.UPDATE_DAILY_FEATURES(150);")
)

```

```

# Tâche de fin

```

```

end_pipeline = EmptyOperator(
    task_id='end_pipeline'
)

```

```

# --- 4. LE CHAÎNAGE DES DÉPENDANCES (Le Dessin du Graphe) ---

```

```
start_pipeline >> task_check_data >> task_run_dataflow >>
task_run_bigquery_kpis >> end_pipeline
```

2. Décryptage des Concepts Clés d'Airflow

Lorsque vous lisez ce code, trois éléments indispensables à la culture d'un Data Engineer doivent sauter aux yeux :

Le bloc `default_args`

C'est le contrat d'assurance de votre pipeline. La ligne 'retries': 2 est essentielle en production. Si un bug réseau temporaire se produit pendant que BigQuery calcule une Window Function, Airflow ne va pas paniquer : il va automatiquement attendre 5 minutes (retry_delay) et relancer la tâche de manière isolée.

Le `catchup=False`

Par défaut, si vous déployez un DAG aujourd'hui (en mai 2026) avec une start_date fixée au 1er janvier, Airflow va essayer de rattraper son retard en lançant instantanément des centaines d'instances (une pour chaque jour manqué). Positionner catchup=False lui indique de ne s'exécuter qu'à partir de maintenant.

L'opérateur de chaînage >>

C'est la syntaxe propre à Airflow. Grâce à la ligne finale, vous définissez l'ordre exact. Si la tâche trigger_dataflow_etl échoue (par exemple, si le code Python de nettoyage plante), le flux s'arrête net. La tâche execute_bigquery_window_functions passera au statut upstream_failed (échec en amont) et ne s'exécutera pas, protégeant ainsi vos tables finales d'un calcul corrompu.

3. Visualisation dans l'interface Airflow

Une fois ce fichier déposé dans l'orchestrateur, le moteur va le compiler et générer une interface graphique :

- Chaque tâche devient une boîte cliquable.
- Le contour de la boîte change de couleur en temps réel selon son statut : Vert (Succès), Rouge (Échec), Jaune (En cours d'exécution).
- En cliquant sur une boîte rouge, vous accédez directement aux logs (print) de cette fonction spécifique pour déboguer en quelques secondes.

Maintenant que vous maîtrisez la logique d'écriture d'un DAG, nous allons remplacer ces simulations Python par les véritables opérateurs GCP natifs pour piloter BigQuery et Dataflow pour de vrai.

H6 : Orchestration BigQuery + Dataflow (mix)

Nous y voilà : c'est le moment d'assembler toutes les pièces du puzzle. Nous allons remplacer les simulations par de véritables opérateurs Google Cloud natifs.

Le but est de créer un DAG de production dans Cloud Composer qui va piloter le scénario MLOps de bout en bout :

1. Déclencher le pipeline de nettoyage Dataflow (Apache Beam) pour ingérer les fichiers JSON de transactions et remplir notre table BigQuery partitionnée.

2. Une fois l'ingestion réussie, exécuter une requête de transformation analytique lourde dans BigQuery (notre calcul de Window Functions) pour mettre à jour nos features de fraude.

1. Le Code du DAG de Production (Mix Cloud)

Voici le script Python complet. Pour que ce fichier fonctionne sur Cloud Composer, il utilise les packages providers de Google.

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.google.cloud.operators.dataflow import
DataflowTemplatedJobStartOperator
from airflow.providers.google.cloud.operators.bigquery import
BigQueryExecuteQueryOperator

# --- 1. CONFIGURATION DES PARAMÈTRES GCP ---

PROJECT_ID = 'fintec-496811'
REGION = 'europe-west1'
BUCKET_NAME = 'fintec-496811-bucket'

# Fichier de configuration du pipeline Beam compilé (Template Dataflow)
DATAFLOW_TEMPLATE_PATH =
f'gs://{BUCKET_NAME}/templates/fraud_detection_template'

# Requête SQL de Feature Engineering
BIGQUERY_SQL_QUERY = """
    CREATE OR REPLACE TABLE
`fintec-496811.fraud_detection.daily_fraud_features` AS
    SELECT
        step,
        type,
        amount,
        nameOrig,
        -- Calcul de la moyenne glissante des transactions par
utilisateur
        AVG(amount) OVER(
            PARTITION BY nameOrig
            ORDER BY step
            ROWS BETWEEN 5 PRECEDING AND CURRENT ROW
        ) as avg_amount_5_tx,
        -- Détection des écarts brutaux par rapport au comportement
habituel
        (amount - AVG(amount) OVER(PARTITION BY nameOrig)) as
deviation_from_avg
    FROM
```

```
`fintec-496811.fraud_detection.raw_transactions_partitioned_clustered`
    WHERE
        -- On ne recalcule les features que sur les données fraîchement
        ingérées (Ex: dernières 24h)
        _PARTITIONTIME >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1
DAY);
"""
```

```
# --- 2. DÉFINITION DU DAG ---
```

```
default_args = {
    'owner': 'mlops_team',
    'depends_on_past': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    dag_id='prod_orchestration_fraud_dataflow_bigquery',
    default_args=default_args,
    description='Orchestration de l ingestion Dataflow et du calcul de
features BigQuery',
    schedule_interval='@daily',
    start_date=datetime(2026, 5, 20),
    catchup=False,
    tags=['prod', 'gcp', 'mlops'],
) as dag:
```

```
# --- 3. LES OPÉRATEURS CLOUD NATIFS ---
```

```
# Tâche 1 : Lancement de l'ETL Apache Beam via Dataflow
trigger_dataflow_etl = DataflowTemplatedJobStartOperator(
    task_id='gcp_dataflow_json_ingestion',
    template=DATAFLOW_TEMPLATE_PATH,
    job_name='dataflow-fraud-ingestion-job',
    location=REGION,
    project_id=PROJECT_ID,
    # Paramètres dynamiques passés au code Apache Beam
    parameters={
        'input_gcs_path':
f'gs://{BUCKET_NAME}/incoming_raw_transactions/*.json',
    },
)
```

```
# Tâche 2 : Calcul des features analytiques dans BigQuery
run_bigquery_features = BigQueryExecuteQueryOperator(
    task_id='gcp_bigquery_feature_engineering',
    sql=BIGQUERY_SQL_QUERY,
    use_legacy_sql=False, # On force l'utilisation du Standard SQL
    location='EU',        # Localisation des données BigQuery
)

# --- 4. LE CHAINAGE DES SERVICES ---

trigger_dataflow_etl >> run_bigquery_features
```

2. Anatomie du flux : Que se passe-t-il en coulisses ?

Lorsque Cloud Composer va exécuter ce DAG, une chorégraphie asynchrone complexe va se mettre en place entre les API de Google :

Étape 1 : Le passage de relais à Dataflow

L'opérateur *DataflowTemplatedJobStartOperator* ne fait pas tourner le code Python sur le serveur d'Airflow. Il envoie un ordre d'allumage léger (un appel API) au service Dataflow de GCP. Une fois l'ordre reçu, Dataflow instancie ses propres Workers (les serveurs de calcul) pour exécuter le pipeline de nettoyage.

- Le comportement d'Airflow : La tâche reste en couleur jaune (en cours). Airflow effectue un "polling" (il demande toutes les 30 secondes à Dataflow : "Tu en es où ?").

Étape 2 : Le signal de fin d'ingestion

Dès que Dataflow a fini de lire les fichiers de transactions et a tout injecté dans la table BigQuery, le job Dataflow passe au statut *SUCCESS*. Airflow le détecte, passe la boîte en vert, et libère le jeton pour l'étape suivante.

Étape 3 : L'activation de la puissance BigQuery

La tâche *BigQueryExecuteQueryOperator* prend le relais. Elle envoie le script SQL de Feature Engineering à BigQuery. BigQuery alloue instantanément ses "Slots" de calcul pour traiter la requête de fenêtrage analytique. Comme notre requête cible la table partitionnée et utilise le filtre *_PARTITIONTIME*, elle ne scanne que les données de la veille, minimisant le coût de la requête.

3. Gestion des pannes en MLOps : Le scénario catastrophe

Imaginons qu'un fichier JSON corrompu fasse planter le pipeline Dataflow (la tâche 1 passe au rouge).

- **Sécurité des données** : Le flux est immédiatement stoppé. La tâche BigQuery ne s'exécutera jamais. Cela évite de polluer ou de fausser l'historique de vos features MLOps avec des données incomplètes.
- **Le réflexe Data Engineer** : Dans l'interface Cloud Composer, vous allez corriger le fichier ou modifier le template, puis faire un "Clear" sur la tâche Dataflow. Airflow va relancer uniquement Dataflow, puis enchaînera automatiquement sur BigQuery une fois le bug résolu.

Data Studio

Data Studio (ex-Looker Studio avec approach Looker & LookML)

- Créer un dashboard Looker Studio connecté à BigQuery
- Utiliser des paramètres, des blended data, des calculs de champs

Après avoir extrait, nettoyé et transformé nos données via Apache Beam et BigQuery, il est temps de rendre ces informations actionnables pour les équipes métiers (analystes fraude, conformité, direction). Pour cela, nous allons utiliser Looker Studio (anciennement Google Data Studio).

Une précision importante : Looker Studio (gratuit, parfait pour des dashboards rapides et connectés à BigQuery) se distingue de la suite payante Looker qui repose sur le langage de modélisation LookML. Nous allons ici nous concentrer sur la version accessible et ultra-efficace : Looker Studio, en exploitant toute sa puissance (paramètres, jointures de données et champs calculés).

Entrons directement dans le vif du sujet avec la première brique : ajouter des graphiques simples pour piloter l'activité de notre FinTech.

1. Connexion initiale : Lier Looker Studio à BigQuery

Pour commencer, la liaison entre Looker Studio et BigQuery se fait de manière totalement native, sans avoir besoin de manipuler des clés API ou des identifiants complexes.

2. Configurer les 3 indicateurs de base (KPI Cards / "Scorecards")

Pour un dashboard de fraude, la première chose qu'un utilisateur veut voir en ouvrant la page, ce sont les chiffres globaux sous forme de cartes d'analyse.

Graphique 1 : Le Volume Total des Transactions

- Type de graphique : Sélectionnez Période (ou Scorecard / Cliché).
- Dimension de période : Si votre table utilise un champ de type Date/Time, Looker Studio le détectera. Sinon, nous utiliserons nos données globales.
- Métrique : Glissez-déposez le champ amount et configurez l'agrégation sur Somme (SUM).
- Formatage : Dans l'onglet Style, affichez des chiffres compacts (ex: 45,2 M€) pour une lecture instantanée.

Graphique 2 : Le Nombre Total de Transactions

- Type de graphique : À nouveau une carte Période.
- Métrique : Sélectionnez n'importe quel identifiant unique (comme nameOrig) ou créez une métrique avec Record Count (Nombre de lignes). Configurez l'agrégation sur Nombre de valeurs (COUNT).

Graphique 3 : Le Taux de Fraude Global

- Type de graphique : Carte Période.
- Métrique : Glissez le champ isFraud. Comme ce champ vaut 1 pour une fraude et 0 sinon, configurez l'agrégation sur Moyenne (AVERAGE).

- Formatage : Changez le type de données de la métrique en Numérique $\rightarrow$ Pourcentage. Vous obtiendrez ainsi directement le ratio de transactions frauduleuses (ex: 0,12%).

3. Ajouter une série temporelle (Évolution de la fraude)

Pour voir si une attaque est en cours ou analyser les tendances, les analystes ont besoin d'une vue chronologique.

- Type de graphique : Graphique de Série Temporelle (Time Series).
- Dimension principale : Le champ step (qui représente les heures/jours écoulés dans PaySim).
- Métrique : isFraud configuré en Somme (pour voir le nombre brut de fraudes par unité de temps) ou amount en Somme pour voir l'impact financier.

4. Ajouter un graphique à barres (Répartition par type de transaction)

Toutes les transactions ne se valent pas. Dans Fintech Fraud System, la fraude se concentre généralement sur des types spécifiques (TRANSFER et CASH_OUT).

- Type de graphique : Graphique à barres horizontales ou verticales.
- Dimension (Axe des X) : Le champ type.
- Métrique (Axe des Y) : Record Count (pour voir le volume d'activité) et ajoutez une deuxième métrique isFraud (Somme) pour superposer les fraudes réelles.

CI/CD DataOps

CI/CD simplifié (GitHub Actions + tests SQL)

- Expliquer ce qu'est une chaîne CI/CD appliquée à la data
- Créer un pipeline GitHub Actions qui teste une requête SQL
- Automatiser un déploiement simple (ex : vue BigQuery, fichier LookML)
- Parler concrètement de bonnes pratiques Data & Analytics

Bienvenue dans ce nouveau chapitre crucial pour la vie d'un projet en production : le CI/CD appliqué au monde de la Data (souvent appelé DataOps).

Si vous avez déjà connu la mauvaise surprise d'un dashboard Looker Studio qui se casse un lundi matin parce que quelqu'un a modifié une colonne dans BigQuery sans prévenir, ou si un pipeline de production a planté à cause d'une virgule oubliée dans un script SQL... alors vous allez adorer le CI/CD.

Comprendre le CI/CD pour la data

1. Le CI/CD, c'est quoi ? (Définition classique)

Dans le logiciel traditionnel, le CI/CD se découpe en deux phases automatisées qui s'activent à chaque fois qu'un développeur propose une modification de code (via une Pull Request sur GitHub) :

- CI (Continuous Integration / Intégration Continue) : On teste automatiquement le code. Est-ce que la syntaxe est bonne ? Est-ce que les tests unitaires passent ? Si un test échoue, on interdit la fusion du code.
- CD (Continuous Deployment / Déploiement Continu) : Si tous les tests de la phase CI sont verts, le code est automatiquement envoyé ("déployé") sur les serveurs de production, sans intervention humaine manuelle.

2. La spécificité de la Data : Pourquoi est-ce plus dur ?

Appliquer le CI/CD à la Data (Data Engineering, Analytics Engineering) est plus complexe que pour une application web classique. Pourquoi ? Parce que le code d'une application est déconnecté de l'état de sa base de données, alors qu'en Data, le code et la donnée sont fusionnés.

Pour valider un changement sur une requête BigQuery ou un modèle de données, vous devez composer avec trois variables mouvantes :

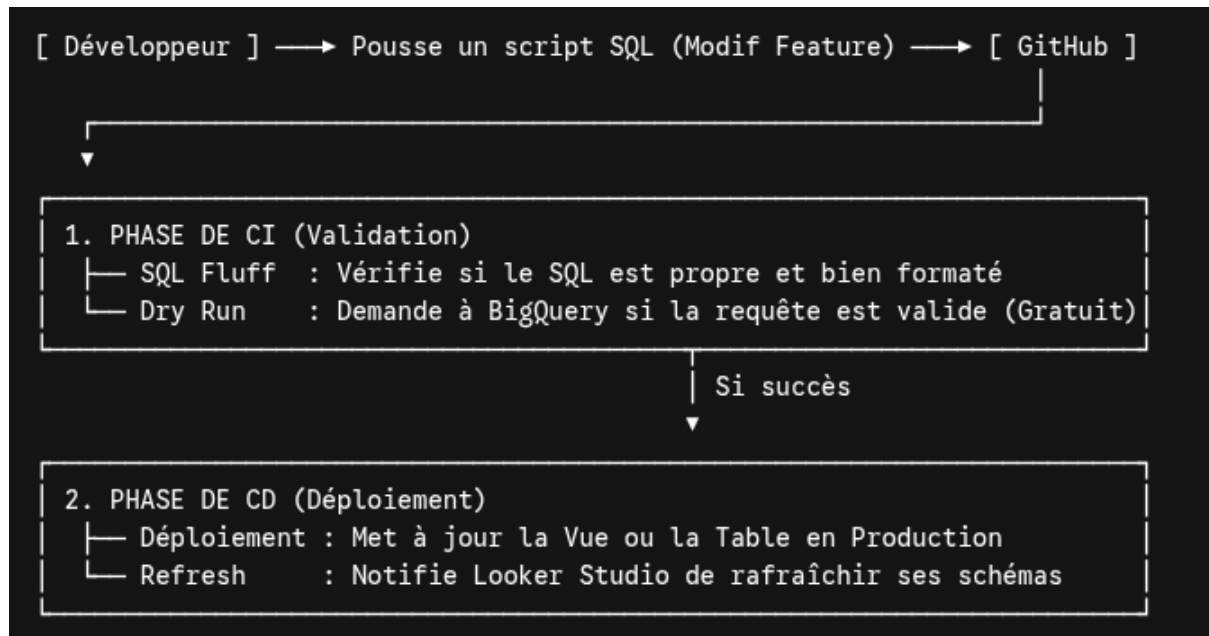
1. Le Code : Votre script SQL, votre pipeline Beam ou vos configurations de dashboards.

2. La Donnée : Le volume, le format et la qualité des lignes stockées dans vos tables.

3. L'Infrastructure : L'entrepôt de données (BigQuery) et ses permissions.

3. À quoi ressemble une chaîne CI/CD Data concrète ?

Pour notre projet de détection de fraude FinTech, imaginez le scénario idéal suivant, entièrement automatisé par un outil comme GitHub Actions :



La Phase de CI (L'inspecteur automatique)

Vous voulez modifier la façon dont on calcule notre moyenne glissante sur les transactions suspects. Vous changez le fichier .sql dans votre dépôt GitHub.

- Le CI se déclenche. Il va lancer un SQL Linter (comme sqlfluff) pour vérifier que vous n'avez pas fait de faute de syntaxe.
- Il va exécuter un Dry Run (une simulation de calcul) directement sur BigQuery pour s'assurer que les tables ou colonnes appelées existent vraiment.

La Phase de CD (Le livreur de confiance)

Si le CI valide votre logique, votre code est fusionné dans la branche principale (main).

- Le CD prend le relais : il se connecte à BigQuery et exécute automatiquement un CREATE OR REPLACE VIEW pour mettre à jour la logique pour toute l'entreprise.
- Aucun humain ne s'est connecté à la console GCP pour faire un copier-coller du script.

4. Les bénéfices du DataOps

- Zéro stress en production : L'erreur humaine est éliminée du processus de déploiement.
- Historique complet (Git) : Si un dashboard affiche un chiffre faux, on peut ouvrir GitHub et voir exactement qui a modifié quelle ligne de SQL, quand, et pourquoi.
- Restauration en 1 clic (Rollback) : Un problème est passé entre les mailles du filet ? On clique sur "Revert" dans GitHub, et la version SQL de la veille est automatiquement réinstallée en production par le pipeline CD.

Créer son premier workflow GitHub Actions

Pour mettre en place notre premier pipeline de CI/CD Data, nous allons utiliser GitHub Actions. C'est l'outil d'automatisation intégré directement à vos dépôts GitHub.

Le but ici est d'écrire un fichier de configuration qui va se déclencher automatiquement à chaque fois qu'un Data Engineer va modifier un script SQL. Ce pipeline va automatiser une étape cruciale du MLOps : le test de validité de la requête auprès de BigQuery (Dry Run) avant d'autoriser sa mise en production.

1. La structure du projet dans GitHub

Dans votre dépôt de code (Repository), GitHub s'attend à trouver vos fichiers d'automatisation dans un dossier au nom très précis. Voici comment structurer vos fichiers :

```
mon-projet-fraude-dataops/
├── .github/
│   └── workflows/
│       └── data_ci.yml      <-- C'est notre fichier de pipeline Actions
└── src/
    └── sql/
        └── update_features.sql <-- Notre script de feature engineering
```

2. Le fichier de Workflow (data_ci.yml)

Voici le code complet à placer dans `.github/workflows/data_ci.yml`. Ce fichier utilise le format YAML. Il décrit les étapes que GitHub va exécuter sur ses serveurs cloud sécurisés.

```
name: Data Analytics CI

# 1. DÉCLENCHEURS (Triggers)
on:
  pull_request:
    branches:
      - main
  paths:
    - 'src/sql/**/*.sql' # Le pipeline s'active UNIQUEMENT si un fichier
                           SQL est modifié

# 2. LES TÂCHES (Jobs)
jobs:
  test_sql_queries:
    runs-on: ubuntu-latest # GitHub fournit une machine Linux vierge

    steps:
      # Étape 1 : Récupérer le code du dépôt Git
      - name: Checkout Code
        uses: actions/checkout@v4

      # Étape 2 : Configurer et s'authentifier sur Google Cloud
```

```

- name: Authenticate to Google Cloud
  uses: google-github-actions/auth@v2
  with:
    credentials_json: ${ secrets.GCP_SA_KEY } # Clé secrète
sécurisée

# Étape 3 : Installer le SDK Google Cloud (gcloud et bq)
- name: Set up Cloud SDK
  uses: google-github-actions/setup-gcloud@v2
  with:
    project_id: 'fintec-496811'

# Étape 4 : Tester la requête SQL via un Dry Run BigQuery
- name: BigQuery Dry Run Test
  run: |
    echo "Vérification de la validité de la requête SQL..."

    # La commande bq avec le flag --dry_run valide la syntaxe et
les colonnes
    # sans exécuter la requête (0 octet scanné, 100% gratuit)
    bq query --use_legacy_sql=false --dry_run <
src/sql/update_features.sql

```

3. Décryptage de la mécanique du pipeline

Que se passe-t-il exactement lorsque ce fichier s'exécute ?

Le filtre de déclenchement (paths)

La ligne - 'src/sql/.sql' est une excellente pratique. Elle évite de faire tourner vos serveurs de tests pour rien. Si vous modifiez simplement le fichier README.md de documentation, GitHub Actions l'ignore. Il ne s'allume que si du code SQL analytique est touché.

La sécurité des accès (secrets.GCP_SA_KEY)

Pour que GitHub puisse demander à BigQuery si votre code est bon, il a besoin d'une autorisation. On ne stocke jamais de mots de passe ou de clés de comptes de service GCP en texte brut dans le code Git. On crée une variable secrète encryptée dans les paramètres de GitHub (Settings -> Secrets and Variables -> Actions) nommée GCP_SA_KEY. GitHub l'injecte de manière invisible au moment du run.

Le super-pouvoir du --dry_run

C'est l'outil ultime de la CI Data. La commande bq query --dry_run envoie le fichier SQL à l'API BigQuery. BigQuery analyse la requête :

- Est-ce que les virgules sont au bon endroit ?
- Est-ce que la table fraud_detection.raw_transactions existe ?
- Est-ce que le champ isFraud est bien épelé ou est-ce que quelqu'un a écrit is_fraud ?

Si tout est correct, BigQuery renvoie un statut OK (ainsi que le volume de données que la requête s'apprête à scanner). Le pipeline passe au vert. S'il y a une erreur, BigQuery renvoie le message d'erreur précis, le pipeline passe au rouge, et la fusion du code est bloquée sur GitHub.

4. Ce que voit l'équipe à la soumission du code

Lorsqu'un membre de l'équipe ouvre une Pull Request pour fusionner ses modifications, GitHub affiche un encadré en bas de la page :

- Pendant le calcul : Une icône jaune tourne (Data Analytics CI / test_sql_queries (run)).
- En cas de succès : Un badge vert indique "All checks have passed", le bouton "Merge" devient accessible.
- En cas d'échec : Un badge rouge bloque le projet. En cliquant sur Details, le développeur voit directement le log de BigQuery : Error: Line 12: Column 'deviation_from_avg' not found. Il sait exactement quoi corriger sans avoir perturbé la production.

```
<FollowUp label="Prêt à automatiser le déploiement de la requête une fois le test réussi (CD) ?" query="Comment compléter ce fichier GitHub Actions pour qu'il déploie automatiquement la requête sous forme de Vue dans BigQuery après la fusion du code ?"/>.
```

Tester une requête BigQuery (avec validation)

Pour que notre pipeline d'Intégration Continue (CI) soit digne d'un environnement de production, nous allons pousser la validation de notre requête BigQuery beaucoup plus loin qu'un simple test de syntaxe.

Dans le monde de la Data, une requête peut être syntaxiquement parfaite pour BigQuery, mais totalement fautive d'un point de vue métier (par exemple : une mauvaise jointure qui multiplie les lignes, ou une division par zéro qui renvoie des valeurs nulles).

Nous allons configurer une étape de validation robuste qui combine deux types de tests :

1. La validation structurelle (Le Dry Run : gratuit et instantané).
2. La validation logique (L'exécution sur un jeu de données de test avec assertions).

1. Le script SQL à tester (update_features.sql)

Imaginons la requête de Feature Engineering que notre équipe souhaite mettre à jour. Elle calcule le taux de fraude par step :

```
-- src/sql/update_features.sql
SELECT
  step,
  COUNT(1) as total_transactions,
  SUM(isFraud) as total_frauds,
  -- Logique métier : taux de fraude
```

```

SAFE_DIVIDE(SUM(isFraud), COUNT(1)) as fraud_rate
FROM
  `fintec-496811.fraud_detection.raw_transactions`
GROUP BY
  step

```

2. Le Workflow de Test Avancé (data_ci.yml)

Nous allons mettre à jour notre fichier GitHub Actions. Après avoir validé le dry-run, la machine virtuelle va exécuter la requête dans un dataset isolé de "Staging" (ou de test) et vérifier des conditions de qualité (assertions).

```

name: Data Analytics CI & Validation

on:
  pull_request:
    branches: [ main ]
    paths: [ 'src/sql/**/*.sql' ]

jobs:
  validate_data_logic:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Authenticate to Google Cloud
        uses: google-github-actions/auth@v2
        with:
          credentials_json: ${ secrets.GCP_SA_KEY }

      - name: Set up Cloud SDK
        uses: google-github-actions/setup-gcloud@v2

      # ÉTAPE 1 : Validation Structurelle (Dry Run)
      - name: BigQuery Dry Run
        run: |
          echo "=== ÉTAPE 1 : Vérification de la structure ==="
          bq query --use_legacy_sql=false --dry_run <
src/sql/update_features.sql

      # ÉTAPE 2 : Exécution de test dans un environnement isolé
      - name: Create Test Materialization
        run: |
          echo "=== ÉTAPE 2 : Matérialisation de test ==="
          # On exécute la requête et on enregistre le résultat dans une
table temporaire de CI

```

```

    bq query --use_legacy_sql=false \
      --destination_table=fraud_detection.ci_test_features \
      --replace=true \
      < src/sql/update_features.sql

# ÉTAPE 3 : Validation Logique (Les Assertions Data)
- name: Run Data Quality Assertions
  run: |
    echo "=== ÉTAPE 3 : Tests de qualité des données ==="

    # Test A : Vérifier que la table de test n'est pas vide
    ROW_COUNT=$(bq query --use_legacy_sql=false --format=csv
"SELECT COUNT(1) FROM fraud_detection.ci_test_features" | tail -n +2)
    if [ "$ROW_COUNT" -eq 0 ]; then
      echo "❌ Erreur : La requête renvoie 0 ligne !"
      exit 1
    fi

    # Test B : Vérifier qu'il n'y a aucune valeur NULL critique
    (ex: fraud_rate)
    NULL_COUNT=$(bq query --use_legacy_sql=false --format=csv
"SELECT COUNT(1) FROM fraud_detection.ci_test_features WHERE fraud_rate
IS NULL" | tail -n +2)
    if [ "$NULL_COUNT" -gt 0 ]; then
      echo "❌ Erreur : Des taux de fraude NULL ont été générés
($NULL_COUNT lignes) !"
      exit 1
    fi

    echo "✅ Tous les tests logiques sont passés avec succès !"

```

3. Pourquoi cette approche sécurise vos Dashboards ?

En appliquant ce triple niveau de sécurité, vous éliminez les régressions avant qu'elles n'arrivent dans Looker Studio :

L'isolation complète

Le pipeline écrit dans *fraud_detection.ci_test_features*. La table de production que vos utilisateurs et vos graphiques regardent n'est jamais touchée durant cette phase. Le test est totalement invisible pour l'entreprise.

Le concept des "Assertions" (Data Quality)

La fin du script utilise de petits scripts Bash combinés à des requêtes SQL rapides pour valider des règles métiers.

- Si une modification dans le code du Data Engineer introduit un bug qui passe le taux de fraude à NULL, l'étape Test B intercepte l'anomalie.

- La commande `exit 1` force GitHub Actions à stopper immédiatement le pipeline et à lever un drapeau rouge.

Économie de ressources

La phase de Dry Run étant placée au tout début, si le développeur a simplement oublié une virgule, le pipeline s'arrête en quelques secondes à l'Étape 1. On n'exécute l'Étape 2 (qui consomme des ressources de calcul sur BigQuery) que si la structure SQL est validée à 100%.

4. Et pour les projets plus volumineux ? (L'alternative dbt)

Si votre architecture SQL commence à comporter des dizaines de fichiers interconnectés, l'écriture d'assertions manuelles en Bash dans GitHub Actions peut devenir lourde.

C'est là que des outils comme dbt (Data Build Tool) ou Dataform (intégré à BigQuery) interviennent. Ils permettent d'écrire ces tests directement en configuration YAML (ex: `not_null`, `unique`) et de lancer un simple dbt test ou dataform test au cours de l'étape de CI de votre workflow GitHub Actions.

Déployer automatiquement une table BigQuery (CD)

Une fois que nos tests d'Intégration Continue (CI) sont passés au vert, nous pouvons aborder l'étape finale du DataOps : le Déploiement Continu (CD).

Le but est d'automatiser complètement la mise en production. Dès qu'un responsable valide et fusionne (merge) la Pull Request sur GitHub, le pipeline va prendre le script SQL validé et mettre à jour la table ou la vue de production dans BigQuery. Plus besoin de se connecter manuellement à la console Google Cloud.

1. La stratégie de déploiement : Vue vs Table

En Data Engineering, deux choix s'offrent à vous pour exposer vos données nettoyées (comme notre calcul de fraude par step) à Looker Studio :

- La Vue SQL (`CREATE OR REPLACE VIEW`) : C'est une table virtuelle. Elle ne stocke pas physiquement les données, elle réexécute la requête à chaque fois qu'un graphique Looker Studio est ouvert. C'est idéal pour des transformations rapides ou des tables légères, car le déploiement prend moins d'une seconde.
- La Table Matérialisée/Incrémentale (`CREATE OR REPLACE TABLE`) : La requête calcule le résultat et l'écrit physiquement sur le disque. C'est indispensable pour les gros volumes de données afin de garantir des performances de filtrage ultra-rapides dans Looker Studio et de maîtriser les coûts de requêtage.

Pour notre exemple de production, nous allons automatiser le déploiement d'une Table de production.

2. Le Workflow de Déploiement Complet (data_cd.yml)

Nous allons créer un nouveau fichier de workflow dans votre dépôt GitHub : `.github/workflows/data_cd.yml`.

Contrairement au fichier de CI, celui-ci ne se déclenche que lorsqu'un changement est

validé et poussé directement sur la branche principale (main).

```
name: Data Analytics CD (Production Deployment)
```

```
# 1. DÉCLENCHEUR : Uniquement lors d'un push sur main
on:
```

```
  push:
    branches:
      - main
    paths:
      - 'src/sql/**/*.sql'
```

```
jobs:
```

```
  deploy_production_data:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Authenticate to Google Cloud
        uses: google-github-actions/auth@v2
        with:
          credentials_json: ${ secrets.GCP_SA_KEY }

      - name: Set up Cloud SDK
        uses: google-github-actions/setup-gcloud@v2
```

```
# 2. ACTIONS DE DÉPLOIEMENT
```

```
- name: Deploy Feature Table to BigQuery Production
  run: |
    echo "🚀 Lancement du déploiement en Production..."
```

```
    # On crée ou remplace la table de production finale avec le
script SQL validé
```

```
    # L'utilisation de l'option --destination_table permet de
matérialiser le résultat
```

```
    bq query --use_legacy_sql=false \
--destination_table=fraud_detection.prod_fraud_features_by_step \
--replace=true \
< src/sql/update_features.sql
```

```
    echo "✅ Table de production mise à jour avec succès !"
```

3. Les Bonnes Pratiques d'Architecture pour le CD Data

Pour que ce pipeline de déploiement automatique reste parfaitement sécurisé et

n'interrompe jamais le service de vos dashboards Looker Studio, voici les règles d'or à appliquer :

L'idempotence (Essentiel)

La commande SQL ou l'appel API utilisé doit être idempotent, c'est-à-dire qu'il doit pouvoir être exécuté 10 fois d'affilée sans provoquer d'erreur et en donnant toujours le même résultat. L'utilisation du flag `--replace=true` avec la commande `bq` garantit que si la table existe déjà, elle est écrasée proprement, et si elle n'existe pas, elle est créée.

La gestion fine des droits d'accès (Moindre privilège)

La clé secrète `${ secrets.GCP_SA_KEY }` utilisée par GitHub correspond à un "Compte de Service" créé dans Google Cloud.

- Pour l'étape de CI (tests), ce compte a uniquement besoin des droits de lecture et d'écriture dans un dataset de test (BigQuery Data Editor sur le dataset `ci_staging`).
- Pour l'étape de CD (déploiement), ce compte doit avoir les droits d'écriture sur le dataset de production (BigQuery Data Editor sur le dataset `production`). Il ne doit jamais être Owner (administrateur) global du projet GCP pour des raisons de sécurité.

Le Blue-Green Deployment (Pour aller plus loin)

Sur des tables de plusieurs téraoctets lues en continu par des milliers d'utilisateurs, écraser brutalement la table de production avec `--replace=true` peut causer une micro-coupure ou ralentir un dashboard pendant la durée du calcul (qui peut prendre plusieurs minutes).

La bonne pratique consiste à :

- Déployer le résultat dans une table temporaire (ex: `prod_fraud_features_by_step_v2`).
- Une fois le calcul terminé et réussi, exécuter une commande de renommage rapide (un "switch") pour remplacer l'ancienne table par la nouvelle de manière totalement invisible pour les utilisateurs finaux.

4. Et pour Looker Studio ?

Looker Studio est nativement connecté à votre table BigQuery. Dès que le pipeline CD termine sa tâche et rafraîchit la table `prod_fraud_features_by_step`, Looker Studio affichera automatiquement les nouvelles données lors du prochain rechargement (ou lorsque l'utilisateur cliquera sur "Actualiser les données" sur le rapport). La chaîne est bouclée, de la modification du code jusqu'à la mise à jour du graphique.